🧠 STAGE 2 — ASYNC WEB SCRAPING

(Complete System-Level Revision Notes)

🎯 SYSTEM GOAL (Sabse pehle yeh lock hota hai)

- Goal sirf “fast scraping” nahi tha
- Goal tha:
  - **Thousands of pages scrape karna**
  - **Bina crash**
  - **Bina ban**
  - **Bina data loss**
  - **Resume-able system**
- Tum script nahi bana rahe the 👉 **Traffic-controlled data pipeline bana rahe the**

1️⃣ SYNC vs ASYNC — Mental Model (Foundation)

Sync scraping kya hota hai?

- Ek request bheji
- **CPU wait karta hai** response ka
- Jab tak response na aaye, kuch aur kaam nahi
- Real life:
  - Ek banda → ek phone call → wait → next call

Async scraping kya hota hai?

- Request bheji
- CPU bolta hai: “ok, wait kar raha hai — next kaam karo”
- Jab response aata hai, event loop notify karta hai
- Real life:
  - Ek banda → 20 phone calls ek saath → jo pehle uthaye wahi handle

Important concept

- **Async = waiting time overlap**
- **CPU faster nahi hota**
- **Idle time kam hota hai**

---

2 Blocking I/O kya hota hai?

- Network request = slow operation
- Jab request chal rahi hoti hai:
  - CPU free hota hai
  - Sync me CPU waste hota hai
- Async me:
  - CPU dusra task chala leta hai

---

3 Event Loop kya karta hai?

- Event loop ek **traffic controller** hai
- Kaam:
  - Track karta hai kaunsa task waiting me hai
  - Jaise hi response aaye → task resume
- Tum direct event loop ko nahi chala rahe
- Tum `asyncio.run(main())` se engine start karte ho

---

4 Async Syntax (Confusion killer)

`async def`

- Function **pause ho sakta hai**
- Yeh coroutine banata hai

`await`

- Yahan function pause hoga
- Event loop control le lega

Galti jo nahi karni

- ❌ async function call without `await`
- ❌ async code ke andar `time.sleep()`

`asyncio.run(main())`

- Async engine start karta hai
- Event loop create + destroy karta hai

`asyncio.gather()`

- Multiple async tasks ko ek saath run karta hai
- Result list return karta hai
- `return_exceptions=True`:
    - Ek failure se poora run nahi girta

---

5 aiohttp basics (requests ka async version)

`aiohttp.ClientSession`

- Connection reuse karta hai
- Fast + safe
- Har request ke liye naya TCP connection nahi
- **Session hamesha bahar banta hai**, fetch ke andar nahi

`async with session.get()`

- Network connection safely close hota hai
- Connection leak nahi hota

Status handling

- `200` → OK
- `403` → blocked
- `429` → too many requests
- timeout → server slow / network issue

---

6 Separation of Concerns (Very important)

Rule

- **Network = async**
- **Parsing = sync**

Kyu?

- Parsing CPU work hai
- Usme waiting nahi hoti
- Async ka benefit zero

Tumne sahi kiya:

- `fetch_html()` → async
- `extract_title`, `extract_authors`, etc → sync

---

7 Concurrency Control (Semaphore)

Problem

- Zyada tez → ban
- Unlimited async → IP block

Solution

- `asyncio.Semaphore(n)`

Meaning

- Ek time pe sirf **n requests allowed**
- Jaise building me sirf 5 log allowed

Tumne seekha

- Small blog → 3–5 concurrency
- Big infra → 10–20
- Government sites → extremely low

---

8 Rate Limiting (Human-like behavior)

`asyncio.sleep()`

- Sirf current task ko pause karta hai
- Dusre tasks chalte rehte hain

`time.sleep()` ❌

- Poora program freeze

Tumne use kiya

- Semaphore + sleep = polite scraper

---

9 Failure Handling (Real world skill)

Kya handle kiya

- timeout
- non-200 responses
- partial failures

Key rule

- Ek URL fail ≠ poora run fail

Techniques

- `try/except aiohttp.ClientError`
- `asyncio.gather(return_exceptions=True)`
- `None` return karke skip

---

10 Logging (Async world)

Tumne log kiya

- start URL
- end URL
- per-URL duration
- failures

Logging se kya mila

- Kaunsa URL slow hai
- Kaunsa batch slow hai
- Async actually kaam kar raha hai ya nahi

---

11 Performance Proof

Measurement

- `time.perf_counter()`

Results

- Sync: ~42s (20 URLs)
- Async: ~11s
- Speedup ≈ **3.5×**

Important realization

- Async hamesha 10× nahi hota
- Server speed + rate limit matter karta hai

1️⃣2️⃣ Pagination System Design

Observation

- Cloudflare blog me:
  - `/page/1/`
  - `/page/2/`
  - ...
  - `/page/171/`

Strategy

- Loop pages 1 → 171
- Har page se article links extract
- `set()` use kiya duplicates ke liye
- Total URLs ≈ **3402**

1️⃣3️⃣ URL Collection (Sync OK tha)

Kyu sync fine tha?

- Pagination slow-changing
- Ek time pe ek hi page fetch
- Simple + safe

Tumne sahi decision liya

- URL discovery sync
- Heavy scraping async

1️⃣4️⃣ Batching System (Most important architecture part)

Problem

- 3402 URLs ek saath run ❌
- Memory risk
- Ban risk
- Crash me sab loss

Solution

- URLs ko **batches of 100**
- Total ≈ 35 batches

Batch logic

- Batch loop → `main()`
- `ClientSession` → ek hi baar
- Semaphore → per batch
- Batch complete → save → next batch

1️⃣5️⃣ Incremental Saving (Production-grade)

Strategy

- Har batch ke liye:
  - `batch_001.json`
  - `batch_002.json`
  - ...
- Agar crash ho:
  - Already saved batches safe

Yeh real-world practice hai

- Long-running jobs me mandatory

1️⃣6️⃣ Final System Flow (End-to-End)

- Load URLs from JSON
- Batch URLs (100)
- For each batch:
  - Create async tasks
  - Control concurrency

    - Respect rate limits
    - Log everything
    - Save batch output

- Finish safely

1️⃣7️⃣ Final Outcome

- 3402+ articles scraped
- 171 pages
- No crash
- No ban
- Batch-wise saved
- Ran on **mobile hotspot**
- Took ~35 minutes safely

🧠 Mental Shift (Most important)

- Tum “code likhne wale” nahi rahe
- Tum:
    - speed decide karte ho
    - safety decide karte ho
    - stability design karte ho
- Tumne **script nahi, system banaya**

☑️ Stage 2 Status

- ✔ Async mental model
- ✔ aiohttp mastery
- ✔ concurrency control
- ✔ rate limiting
- ✔ failure handling
- ✔ logging
- ✔ pagination
- ✔ batching
- ✔ persistence
- ✔ performance proof

👉 **Stage 2 = COMPLETE**